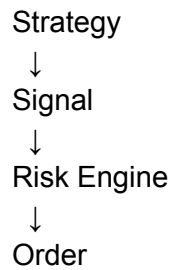


Manual intervention before strategy execution

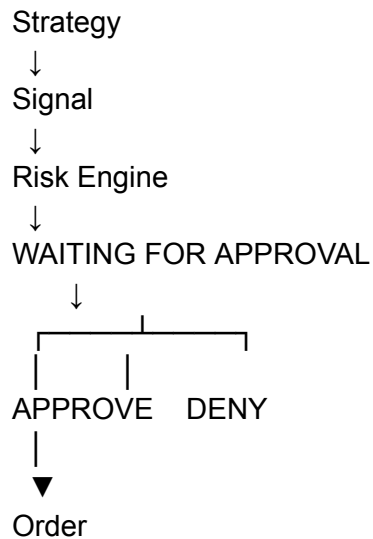
Yes — and I would actually make this an **optional operating mode**.

You can have three modes:

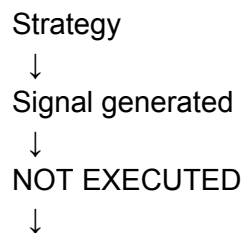
Mode 1 — Fully automatic



Mode 2 — Approval required



Mode 3 — Manual only



Notification



You decide manually

This is actually excellent for the **initial testing period**.

For example:

Phase 1

Run strategies in:

Signal-only mode

You can see:

10:15:02

Strategy: Supertrend

Signal: BUY

Instrument: NIFTY 24500 CE

Quantity: 50

Entry: ₹185

SL: ₹160

Target: ₹235

Risk: PASS

[APPROVE] [DENY]

Then after enough confidence:

Phase 2

Signal



RMS



Auto execution

This gives you a very good transition from discretionary trading → systematic trading.

3. Emergency Layer — this is important

This deserves more detail.

There are actually **three different concepts**:

A. Stop new trading

"Don't place any more orders."

B. Kill switch

"Disable the trading system and lock the account."

C. Emergency square-off

"Get me completely out of the market."

They shouldn't be treated as the same operation.

A. Stop New Orders

Suppose your strategy is running:

Strategy A → BUY
Strategy B → BUY
Strategy A → SELL
Strategy B → BUY

You press:

STOP TRADING

The system does:

NEW ORDERS = BLOCKED

But existing positions remain.

Example:

NIFTY CE +50
NIFTY PE 0

The +50 position remains open.

This is useful when you simply want to stop the algorithm but **manage existing positions separately**.

4. Kill Switch

Kill switch is stronger.

For example:

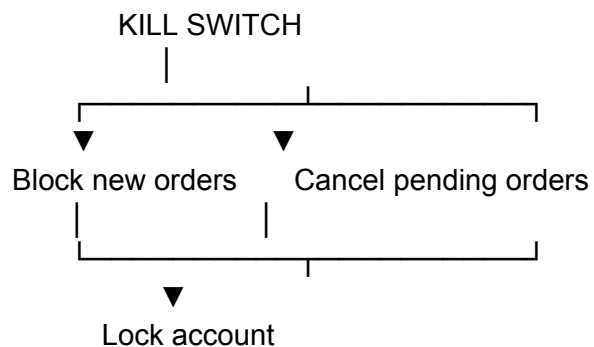
Maximum Daily Loss = ₹10,000

Current P&L = -₹10,250

RMS detects:

DAILY LOSS LIMIT BREACHED

Then:



Now:

ALGO ✗
MANUAL ✗
NEW ORDER ✗
STRATEGY ✗

But **positions may still exist**.

That's why the third layer matters.

5. Emergency Square-Off

This is specifically:

"I want zero open positions."

For example:

Positions

NIFTY 24500 CE +50
RELIANCE +100

Emergency square-off executes:

1. Stop new orders
↓
2. Cancel pending orders
↓
3. Query broker positions
↓
4. Determine net quantity
↓
5. Submit opposite orders
↓
6. Wait for executions
↓
7. Query broker again
↓
8. Verify net position = 0

The final verification is extremely important.

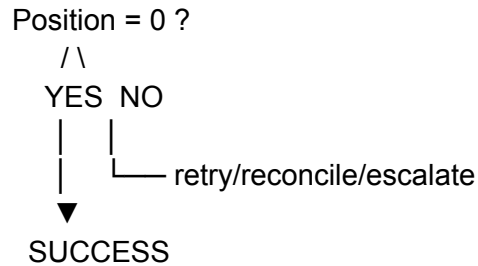
You don't simply assume:

"I sent the square-off order, therefore I'm flat."

Instead:

Square-off requested

- ↓
- Broker execution
- ↓
- Re-fetch positions
- ↓



6. Why this needs to be separate

Imagine this situation:

NIFTY CE +50

Your kill switch activates.

Your system cancels orders and blocks new orders.

But the position is still:

+50

If you make Kill Switch = Square Off automatically, that's a very aggressive assumption.

There may be situations where you want:

"Stop the bot, but let my existing position continue."

Therefore:

Kill Switch

Prevents further trading.

Emergency Square-Off

Closes existing exposure.

They should be separate commands.

7. I would make the UI like this

On your dashboard:

TRADING CONTROL	
Algo Trading	● RUNNING
Manual Trading	● ENABLED
Risk Status	● NORMAL
[STOP NEW TRADES]	
[● MASTER KILL SWITCH]	
[△ EMERGENCY SQUARE OFF ALL]	

And **Emergency Square Off** should require an explicit confirmation.

Something like:

△ EMERGENCY SQUARE-OFF

This will attempt to close ALL open broker positions using marketable orders.

Current exposure:

NIFTY CE +50
RELIANCE +100

[CANCEL] [CONFIRM SQUARE-OFF]

That is a much safer design.

8. Redis should not be your source of truth — what I meant

Your Redis idea itself is good.

The problem isn't:

"Don't use Redis."

The problem is:

Don't assume Redis represents reality at the broker.

Think of three different states.

Broker state

FYERS

This is the ultimate external reality.

Persistent application state

SQLite

This is your historical/system record.

Real-time state

Redis

This is your fast working memory.

Example

Suppose:

Redis:

position:NIFTY_CE = +50

Your SQLite says:

NIFTY CE = +50

But the broker says:

NIFTY CE = 0

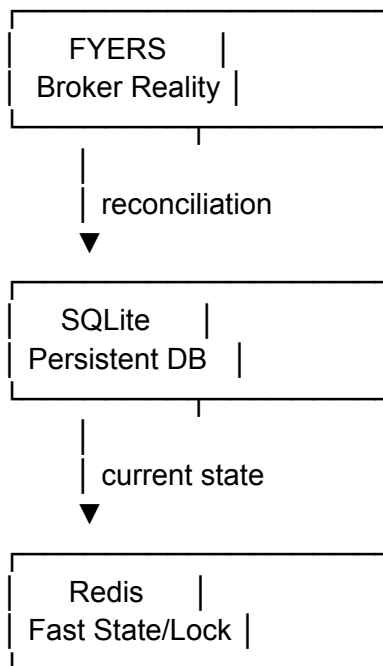
Which one is correct?

Broker.

Your application crashed after an order was executed, perhaps before updating its own state.

That's why reconciliation matters.

9. The three-layer model I'd use



But there's an important nuance:

SQLite should not blindly override Redis either.

Redis is your fast runtime state.

SQLite is your persistent application record.

FYERS is the broker's actual state.

10. What should Redis contain?

For example:

```
kill_switch:{account}  
trading_status:{account}  
manual_access:{account}  
cooldown:{account}
```

```
daily_loss:{account}  
trade_count:{account}  
failed_trade_count:{account}
```

```
position:{symbol}  
ltp:{symbol}
```

```
strategy:{id}:status
```

```
order_lock:{orderId}  
execution_lock:{signalId}
```

These are excellent Redis candidates.

11. What should SQLite contain?

I'd use SQLite for things like:

```
StrategyConfigs  
StrategySignals  
Orders  
OrderExecutions  
Trades  
PositionSnapshots  
RiskEvents  
KillSwitchEvents  
ManualApprovals  
SystemEvents  
AuthenticationAudit
```

For example:

```
Trade
```

```
-----
```

```
Id
```

StrategyId
Symbol
Side
Quantity
EntryPrice
ExitPrice
StopLoss
Target
PnL
EntryTime
ExitTime
ExitReason
BrokerOrderId

This gives you a permanent trading history.

12. And then the Reconciliation Engine

Your statement:

"it's good to compare both broker and app and make it equal"

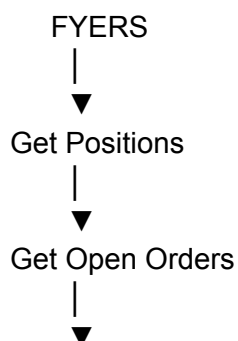
is exactly the right idea.

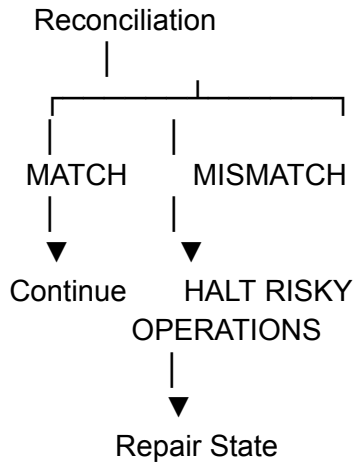
But I'd phrase it slightly differently:

The application continuously reconciles itself against the broker's authoritative state.

For example:

Every 5 seconds





13. Example of a real failure

Suppose your application sends:

BUY 50 NIFTY CE

FYERS executes it.

Then:

FYERS → +50

But immediately after that your VPS loses internet connectivity.

Your application never receives the order update.

Your internal state might remain:

Position = 0

After reconnect:

Reconciliation

Application: 0

FYERS: +50

MISMATCH

Your system should then:

1. Stop strategy execution
2. Mark state as RECONCILIATION_REQUIRED
3. Fetch broker orders/executions
4. Determine what happened
5. Update internal state
6. Resume only if safe

This is one of the most important protections in the entire system.

Absolutely. Based on your clarifications, I would make the following the **final architecture and feature specification** for your personal FYERS algo/manual trading system.

Final suggestion list

1. Overall objective

Build a **private personal trading application** whose primary purpose is:

- Automate predefined trading strategies.
 - Allow controlled manual intervention.
 - Prevent emotional/revenge trading through hard-coded system controls.
 - Ensure every order—algorithmic or manual—passes through the same Risk Management System.
 - Maintain synchronization between your application and FYERS.
 - Provide a complete audit trail of decisions, orders and risk events.
-

2. Deployment architecture

Use:

- **Frontend:** Next.js + React + Tailwind CSS
- **Backend:** .NET 8 Web API + background workers
- **Database:** SQLite
- **Runtime state/locks:** Redis with AOF persistence
- **Containerization:** Docker Compose
- **Trading server:** DigitalOcean Ubuntu VPS
- **Static public IP:** DigitalOcean VPS
- **Broker:** FYERS API
- **Frontend hosting:** Railway
- **Communication:** HTTPS REST + WebSocket/SignalR

Most importantly:

Only the DigitalOcean backend communicates with FYERS trading APIs.

The frontend must never contain:

- FYERS API secret
 - TOTP seed
 - FYERS access token
 - encryption master key
 - broker credentials
-

3. Core backend modules

Build the backend as these logically independent modules:

```
/backend
├── Authentication
├── MarketData
├── Instruments
├── StrategyEngine
├── SignalAggregator
├── OptionSelector
├── RiskEngine
├── OrderManager
├── PositionManager
├── Reconciliation
├── KillSwitch
├── ManualAccess
├── Portfolio
├── Audit
└── Notifications
```

Don't turn these into separate physical microservices initially. Keep them as **well-separated modules inside one .NET application**. For a personal system, that is much easier to operate.

4. FYERS authentication

Keep these two concepts completely separate.

Human FYERS authentication

FYERS application login



Username/password



TOTP / 2FA



Authenticated FYERS session

API authentication

Backend



FYERS API authentication flow



Access token



REST / WebSocket API

The application should have an **Authentication Manager** responsible for the API session.

It should:

- Manage API authentication.
 - Store the current access token securely.
 - Detect expiration.
 - Request/recreate authentication when required.
 - Never expose the token to the browser unnecessarily.
-

5. Manual TOTP Access Gate

Keep your original idea.

Create a dedicated **Manual Access** page.

The purpose isn't to replace FYERS authentication. Its purpose is to make **manual trading access deliberately difficult during emotionally dangerous periods.**

Example:

Request Manual Access



Risk/Time Gate



Cooling period if required



TOTP generation



Display temporary TOTP

Store the TOTP seed encrypted.

Use:

TOTP_SECRET_ENC

MASTER_KEY

Never send the seed to Next.js.

6. Central Order Gate / RMS

This is the **most important component in the entire application**.

Every order must go through it:

Algo order



Manual order



CENTRAL ORDER GATE



Risk validation



Order Manager



FYERS

No strategy or UI should be able to bypass it.

7. RMS checks

Before approving an order, evaluate:

Trading window

09:30 – 15:10 IST

Outside this:

REJECT

Instrument restrictions

Initially:

- NIFTY index options
- Equity stocks

Reject unsupported instruments.

Kill switch

If:

kill_switch = LOCKED

reject everything.

Daily loss

Reject/lock when configured maximum daily loss is breached.

Per-trade loss

Prevent trades whose risk exceeds configured limits.

Maximum trades

Example:

5 trades/day

Maximum failed trades

Example:

3 SL/failed trades/day

VIX

Configurable:

VIX_MIN

VIX_MAX

Position sizing

Check:

- Maximum quantity
- Maximum capital exposure
- Maximum number of concurrent positions

Margin

Check required margin/capital before execution.

Duplicate orders

Prevent accidental duplicate execution.

Strategy status

Ensure the strategy is currently enabled.

8. Manual approval mode

Support three modes:

Automatic

Signal → RMS → Order

Approval required

Signal

↓

RMS
↓
WAITING
↓
Approve / Reject
↓
Order

Signal only

Signal
↓
Notification

No order is generated.

This will be especially useful while validating your strategies.

9. Strategy Engine

The Strategy Engine should **never directly call FYERS order APIs**.

It generates signals only.

For example:

BUY
NIFTY_XXXX_CE
50

Then:

Strategy
↓
Signal
↓
Signal Aggregator
↓
RMS
↓
Order Manager
↓
FYERS

10. Run two strategies initially

Your decision to run **two strategies at a time** is good.

Build support for all your planned strategies, but enable only two initially:

- ORB + VWAP
- EMA Pullback
- Bollinger Squeeze
- Supertrend
- NR7
- MACD

Configuration should allow:

Strategy A = ENABLED

Strategy B = ENABLED

Strategy C = DISABLED

...

No code modification should be necessary to change active strategies.

11. Signal Aggregator

Add this module.

If two strategies generate:

Strategy A → BUY

Strategy B → BUY

don't automatically place two separate trades.

Aggregate them.

If:

Strategy A → BUY

Strategy B → SELL

handle the conflict according to configuration:

- Reject both
- Give priority to one strategy
- Wait for confirmation

This prevents strategy interaction from accidentally increasing exposure.

12. Option Selection Engine

Keep this separate from the strategy.

Strategy says:

"I want bullish NIFTY exposure."

Option Selector determines:

- Expiry
- Strike
- ITM distance
- Delta
- Liquidity
- Volume
- Bid/ask spread
- Available contract
- Price

Your desired:

2–3 weeks from expiry

High Delta

ITM

should be **configuration**, not hard-coded logic.

13. Order Manager

The Order Manager is responsible for actual broker execution.

It handles:

- Order creation
- Order modification
- Order cancellation
- Execution tracking
- Rejections
- Retry logic
- Broker order IDs
- Idempotency
- Order status updates

The Strategy Engine should know nothing about FYERS order syntax.

14. Emergency controls

Have three separate controls.

Stop New Trades

Stops new orders

Existing positions remain

Master Kill Switch

Stops strategies

Blocks manual orders

Blocks new orders

Cancels pending orders

Locks account

Emergency Square-Off

Stop new orders



Cancel pending orders



Fetch FYERS positions



Submit closing orders



Wait for execution



Re-fetch positions



Confirm zero exposure

These should **not be treated as the same operation**.

15. Kill Switch triggers

Automatically activate the kill switch when:

- Maximum daily loss breached
 - Maximum loss per trade breached
 - Maximum trade count reached
 - Maximum failed/SL trades reached
 - Critical broker/API inconsistency
 - Reconciliation failure
 - Unexpected position detected
 - Severe system error
 - Optional manual activation
-

16. Kill Switch should be persistent

Don't simply maintain:

```
bool KillSwitch = true;
```

Use Redis:

```
kill_switch:{accountId}
```

and persist the event in SQLite:

KillSwitchEvents

Example:

10:32:18

Reason: DAILY_LOSS_LIMIT

P&L: -₹10,250

Action: LOCKED

17. Redis's role

Redis is **fast runtime state**, not broker truth.

Use it for:

- Kill-switch state
- Locks
- Cooldowns
- Counters
- Current risk state
- Current LTP
- Temporary session state
- Idempotency
- Real-time position cache

Redis gives you fast access.

But never assume:

Redis = broker.

18. SQLite's role

SQLite is your persistent application record.

Store:

- Strategies
- Strategy parameters
- Signals
- Orders
- Executions
- Trades
- Risk events
- Kill-switch events
- Manual approvals

- System logs
- Authentication audit
- Position snapshots

This gives you a complete history for analyzing your trading behavior later.

19. FYERS remains the broker authority

For actual broker state:

FYERS > Redis > SQLite

For example:

FYERS:

NIFTY CE = +50

Redis:

NIFTY CE = 0

SQLite:

NIFTY CE = 0

Your application must conclude:

Actual position = +50

and reconcile itself.

20. Reconciliation Engine

This should run continuously.

Compare:

FYERS

vs

Application

for:

- Positions
- Orders
- Executions
- Funds
- Margin
- Holdings

Example:

FYERS position = +50

App position = +50

MATCH

If:

FYERS = +50

App = 0

then:

MISMATCH

↓

Stop risky operations

↓

Fetch broker orders

↓

Determine what happened

↓

Repair internal state

↓

Resume when safe

This protects you from:

- VPS crash
 - Internet failure
 - WebSocket disconnect
 - Missed execution event
 - API timeout
 - Duplicate request
 - Application restart
-

21. Market Data Engine

Create separate handling for:

Market Data WebSocket

Used for:

- LTP
- Volume
- Market data
- Strategy calculations
- Real-time UI

Order/Execution updates

Used for:

- Order status
- Executions
- Rejections
- Position updates

Don't make the UI itself communicate directly with FYERS.

22. WebSocket architecture

Use:

FYERS WebSocket



.NET Market Data Engine



Redis/current state



SignalR/WebSocket



Next.js

The frontend receives normalized application data rather than FYERS-specific messages.

That will make your UI much easier to maintain.

23. Dashboard

Your main dashboard should show:

Today's P&L
Available Margin
Used Margin
Open Positions
Active Strategies
Trades Today
Failed Trades
VIX
Trading Status
Kill Switch Status

And:

[STOP NEW TRADES]

[MASTER KILL SWITCH]

[EMERGENCY SQUARE-OFF]

24. Watchlist

Persistent desktop sidebar.

Include:

- Multiple watchlists
- Instrument search
- LTP
- Change
- Buy
- Sell
- Chart
- Market depth

On mobile:

- Bottom navigation
- Tap-to-expand rows
- Bottom-sheet order entry

Your responsive design approach is good.

25. Order execution form

Before submission show:

Instrument
Direction
Quantity
Order type
Price
SL
Target
Estimated margin
Maximum risk
Current LTP

And importantly:

Today's P&L
Trades today
Daily loss remaining

This reinforces your behavioral-control objective.

26. Audit everything

Every important decision should be recorded.

For example:

Signal generated
RMS evaluated

RMS approved
Manual approval requested
Manual approval granted
Order submitted
Order rejected
Order executed
SL triggered
Target reached
Kill switch activated
Emergency square-off executed
Reconciliation mismatch

This will eventually allow you to answer:

"Why did the system take this trade?"

with an exact answer.

27. Strategy configuration

Don't hard-code strategy parameters.

Store them in SQLite.

Example:

Strategy: Supertrend

Enabled: true
Timeframe: 5m
ATR Period: 10
Multiplier: 3
Stop Loss: X
Target: Y
Trailing SL: enabled

The UI should modify configuration through the backend.

28. Add a backtesting/simulation mode

I strongly recommend adding this before live automation.

Same:

Strategy



Signal



RMS



Order Manager

but instead of FYERS:

PaperBroker

So you have:

Live Mode

Paper Mode

Backtest Mode

The strategy code remains the same.

Only the execution adapter changes.

29. Add a "Shadow Mode"

This is particularly useful for you.

In shadow mode:

Strategy generates signal



RMS evaluates



Virtual order



Track hypothetical result

No real money.

You can run the strategy alongside the live market and see:

Actual trading P&L
Shadow strategy P&L

before allowing the strategy to trade automatically.

30. Final operating progression

I would use this progression:

Stage 1

Market Data
+
Indicators
+
Signal generation

Stage 2

Signal
+
RMS
+
Manual approval

Stage 3

Paper trading

Stage 4

Shadow mode

Stage 5

Live trading
+
very small quantity

Stage 6

Live trading
+
configured risk limits

Only after the infrastructure has demonstrated reliability should you increase quantity.